

챕터 2. 동기식 MSA 구현 - 서비스를 연결하다

이 챕터의 전체 소스코드는 <https://github.com/metacoding-12-msa/ex01> 에서 확인할 수 있습니다.

이번 챕터가 끝나면

- Spring Boot로 4개 서비스를 독립적으로 실행하고 REST로 연결할 수 있습니다.
- JWT 인증 필터를 구현하고 서비스 간 Authorization 헤더를 전달할 수 있습니다.
- 주문 생성 시 재고 감소 → 배달 생성 흐름을 동기적으로 구현할 수 있습니다.
- 중간에 실패가 발생했을 때 이전 작업을 되돌리는 보상 트랜잭션을 작성할 수 있습니다.

준비하기. 실습 시작 전 한 번만 설정

1. 소스 코드 클론

터미널 레포 클론

```
git clone https://github.com/metacoding-12-msa/ex01.git
cd ex01
```

2. 파일 구조

ex01 디렉토리

```
ex01/
├─ user/           # 포트 8083
├─ product/        # 포트 8082
├─ order/          # 포트 8081
├─ delivery/       # 포트 8084
└─ docker-compose.yml # 전체 서비스 실행
```

각 서비스 내부는 동일한 구조입니다. 주문 서비스 기준으로 보여드리며, 회원/상품/배달 서비스도 같은 구조입니다.

주문 서비스 패키지 구조

```
src/main/java/com/metacoding/order/
├── OrderApplication.java           # [참고]
├── core/
│   ├── config/
│   │   ├── WebConfig.java         # [참고] JWT 필터 등록
│   │   └── RestClientConfig.java  # [작성] JWT 헤더 전달 인터셉터
│   ├── filter/
│   │   └── JwtAuthenticationFilter.java # [참고] JWT 인가 필터
│   ├── handler/
│   │   ├── GlobalExceptionHandler.java # [참고] 전역 예외 처리
│   │   └── ex/                     # 커스텀 예외 (Exception400~500)
│   └── util/
│       ├── JwtProvider.java        # [참고] JWT 파싱/검증
│       ├── JwtUtil.java            # [참고] JWT 생성
│       └── Resp.java               # [참고] 표준 응답 래퍼
├── orders/
│   ├── Order.java                 # [참고] JPA 엔티티
│   ├── OrderStatus.java           # [참고] 주문 상태 enum
│   ├── OrderController.java        # [참고] REST 컨트롤러
│   ├── OrderService.java           # [작성] 비즈니스 로직
│   ├── OrderRepository.java        # [참고] Spring Data JPA
│   └── OrderRequest.java / OrderResponse.java # [참고]
├── adapter/                       # 주문 서비스에만 존재
│   ├── ProductClient.java          # [작성] 상품 서비스 호출
│   ├── DeliveryClient.java         # [작성] 배달 서비스 호출
│   └── dto/                        # 어댑터용 DTO (ProductRequest, DeliveryRequest)
└── Dockerfile                     # [참고] Docker 이미지 빌드
```

회원/상품/배달 서비스는 `adapter/` 패키지와 `RestClientConfig` 가 없고, 나머지 구조는 동일합니다. 각 서비스의 도메인 패키지명은 복수형(`users/`, `products/`, `deliveries/`)으로 통일합니다.

3. 실습 환경

도구	용도	비고
Docker Desktop	4개 서비스를 컨테이너로 실행	https://www.docker.com/products/docker-desktop/
Hoppscotch	API 호출 결과 확인	https://hoppscotch.io/ (설치 불필요, 브라우저 확장만 추가)

Docker Desktop을 실행하고 "Engine running" 상태인지 확인합니다. 자세한 사용법은 2.7절에서 다룹니다.

4. 실습 순서

1. 공통 설정(JWT·표준 응답·예외 처리)을 `core/` 패키지에 두기
2. 회원·상품·배달 서비스의 핵심 코드 살펴보기
3. 주문 서비스에 RestClient + 보상 트랜잭션 작성하기
4. Docker Compose로 4개 서비스를 한 번에 띄우고 시나리오 3개 검증하기

이번 챕터의 단순화 가정. 학습 흐름을 깔끔하게 가져가기 위해 한 건의 주문에 상품 한 개를 담는 것으로 가정합니다. 실무에서는 `OrderItem` 을 분리하여 1:N 관계로 모델링하는 것이 일반적이지만, 이 책의 핵심인 **분산 트랜잭션 보상 흐름**을 보여주는 데에는 단일 상품 모델로 충분합니다.

오픈이 : "선배님, 서비스 네 개로 나누는 건 알겠는데, 이걸 어떻게 코드로 옮겨요?"

선배 : "일단 각자 따로 띄워서 REST로 연결해 봐요. 서로 전화를 거는 것처럼 직접 호출하는 거예요. 제일 단순한 방법이죠."

2.1 이야기의 시작 - 네 개의 서비스가 만나다

챕터 1에서 설계한 네 개의 서비스(회원, 상품, 주문, 배달)를 이제 직접 만들어 보겠습니다.

이번 챕터의 주인공은 **주문 서비스**입니다. 주문을 생성하려면 상품 서비스에서 재고를 줄이고, 배달 서비스에서 배달을 만들어야 합니다. 즉 주문 서비스가 두 서비스를 직접 호출해야 합니다. 그런데 여기서 중요한 문제가 생깁니다. "재고는 줄였는데 배달 생성이 실패하면 어떻게 해야 할까?"

이 질문에 대한 답이 바로 이번 챕터의 핵심, **보상 트랜잭션**입니다.

보상 트랜잭션(Compensating Transaction)이란? 여러 서비스에 걸친 작업 중 일부가 실패했을 때, 이미 완료된 작업을 되돌리기 위해 역순으로 실행하는 취소 작업입니다.

각 서비스는 독립된 Gradle 프로젝트입니다. 하나의 서비스를 배포할 때 다른 서비스를 건드릴 필요가 없습니다. 디렉토리 구조와 패키지 구조는 준비하기에서 미리 살펴보았습니다. `core/` 패키지에는 JWT, 예외 처리, 표준 응답 등 모든 서비스에 공통으로 필요한 코드가 들어갑니다.

2.2 공통 설정 - 모든 서비스가 공유하는 뼈대

서비스 구조를 잡았으니, 먼저 공통 기반을 만들겠습니다. MSA에서는 서비스마다 서버가 다르므로 세션을 공유할 수 없습니다. 대신 JWT 토큰을 발급하고, 각 서비스가 토큰만 검증하는 방식을 사용합니다.

4개 서비스 모두 JWT 인증, 표준 응답 형식, 예외 처리가 필요합니다. 이를 `core/` 패키지에 모아 두면, 각 서비스는 비즈니스 로직에 집중할 수 있습니다.

각 컴포넌트의 역할은 다음과 같습니다.

컴포넌트	역할
<code>JwtAuthenticationFilter</code>	매 요청마다 <code>Authorization</code> 헤더에서 JWT를 꺼내 검증하고, 토큰 안의 <code>userId</code> 를 request attribute에 저장합니다. 컨트롤러는 <code>@RequestAttribute("userId")</code> 로 현재 사용자를 식별합니다.
<code>JwtUtil / JwtProvider</code>	<code>JwtUtil</code> 은 로그인 성공 시 JWT를 생성하고, <code>JwtProvider</code> 는 요청에서 받은 토큰을 파싱·검증합니다.
<code>Resp</code>	모든 API 응답을 <code>{status, msg, body}</code> 형태로 통일하는 래퍼 클래스입니다. 성공과 실패 모두 같은 구조로 반환하여 클라이언트 파싱을 단순화합니다.
<code>GlobalExceptionHandler</code>	<code>@RestControllerAdvice</code> 로 전역 예외를 잡아 <code>Resp</code> 형태의 에러 응답을 반환합니다.
<code>WebConfig</code>	JWT 필터를 등록합니다. <code>/api/*</code> 경로에 <code>JwtAuthenticationFilter</code> 를 적용하여 인증이 필요한 요청을 필터링합니다.

공통 설정이 준비되었습니다. 주문 서비스의 보상 트랜잭션을 직접 작성하기 전에, 나머지 세 서비스의 핵심 코드를 먼저 살펴보겠습니다. 아래 2.3~2.5는 [\[참고\]](#) 코드로, 동작 이해를 위해 주요 부분만 보여줍니다.

2.3 회원 서비스 - JWT로 로그인하다

회원 서비스는 로그인과 사용자 조회를 담당합니다. 사용자가 `POST /login` 으로 아이디와 비밀번호를 보내면, 회원 서비스가 DB에서 조회하고 비밀번호를 검증합니다. 검증에 성공하면 JWT 토큰을 응답 헤더에 넣어 돌려줍니다. 이 토큰이 이후 모든 서비스 요청의 인증 수단이 됩니다.

메서드	경로	기능
POST	/login	로그인 (JWT 발급)
GET	/api/users/{userId}	사용자 조회

`username` 은 유니크 제약으로 중복 가입을 방지합니다. 더미 데이터는 ssar, cos, love 세 계정이 H2 in-memory DB에 자동 삽입됩니다. 엔티티·서비스·SQL 등 자세한 코드는 GitHub 레포 [ex01/user/](#) 참조.

2.4 상품 서비스 - 재고를 관리하다

상품 서비스는 상품 목록 조회와 재고 증감을 담당합니다. 주문 서비스가 주문을 생성할 때 이 서비스의 재고 감소 API를 호출하고, 주문이 취소되거나 실패하면 재고 증가 API로 되돌립니다.

메서드	경로	기능
GET	/api/products/{productId}	상품 조회
PUT	/api/products/{productId}/decrease	재고 감소 (보상 시 호출됨)
PUT	/api/products/{productId}/increase	재고 증가 (보상 시 호출됨)

재고 감소 시 상품 존재 여부와 가격 일치를 검증합니다. 더미 데이터는 MacBook Pro(재고 10), iPhone 15(재고 0, 품절), AirPods(재고 10)이며, 2.7 시나리오에서 품절 상품으로 주문해 보상 트랜잭션을 확인합니다. 엔티티·서비스·SQL 등 자세한 코드는 GitHub 레포 [ex01/product/](#) 참조.

2.5 배달 서비스 - 배달을 생성하고 취소하다

배달 서비스는 배달 생성과 취소를 담당합니다. 이번 챕터에서는 배달 생성과 동시에 완료 처리합니다. PENDING 상태로 생성한 뒤 즉시 COMPLETED로 전이하므로, 실질적으로 COMPLETED 또는 CANCELLED 두 상태만 관찰됩니다.

메서드	경로	기능
POST	/api/deliveries	배달 생성 + 즉시 완료
GET	/api/deliveries/{deliveryId}	배달 조회
PUT	/api/deliveries/{orderId}	배달 취소 (보상 시 호출됨)

배달 취소는 주문 실패 시 보상 호출로 들어오며, 이미 취소된 배달에 대한 중복 요청은 방어합니다. 엔티티·서비스·SQL 등 자세한 코드는 GitHub 레포 [ex01/delivery/](#) 참조.

회원, 상품, 배달까지는 각자 제 일만 하면 된다. 진짜 문제는 이것들을 엮는 주문 서비스다.

오픈이 : "재고를 줄인 다음에 배달 만들다가 실패하면요?"

선배 : "그때를 대비해서 되돌리는 코드를 직접 짜는 거예요. 어디까지 진행됐는지 기록해 뒀다가, 실패하면 역순으로 취소해요."

2.6 주문 서비스 - 보상 트랜잭션의 현장

배달 서비스까지 살펴봤으니, 이제 핵심인 주문 서비스로 넘어갑니다.

메서드	경로	기능
POST	/api/orders	주문 생성
GET	/api/orders/{orderId}	주문 조회
PUT	/api/orders/{orderId}	주문 취소

먼저 주문과 주문 상품을 표현하는 엔티티부터 살펴보겠습니다.

2.6.1 Order 엔티티

주문은 사용자(`userId`)와 상품 한 건(`productId`, `quantity`, `price`)을 담습니다. 상태는 `PENDING → COMPLETED` 또는 `PENDING → CANCELLED`로 전이됩니다.

필드	타입	역할
id	int	PK, AUTO_INCREMENT
userId	int	주문자
productId, quantity, price	int, int, Long	단일 상품 정보
status	OrderStatus	PENDING / COMPLETED / CANCELLED
createdAt, updatedAt	LocalDateTime	타임스탬프

상태 전이는 `create()`, `complete()`, `cancel()` 메서드로 캡슐화되어 있습니다. 전체 코드는 GitHub 레포 `ex01/order/.../orders/Order.java` 참조.

2.6.2 더미 데이터

사용자별 주문 3건(완료·취소·대기)을 등록합니다. 한 행에 상품 정보까지 함께 들어갑니다. 상세 SQL은 GitHub 레포 `ex01/order/.../db/data.sql` 참조.

2.6.3 OrderRequest, OrderResponse

주문 API의 요청·응답 DTO입니다. 둘 다 `record`로 정의되어 있습니다.

OrderRequest — 요청은 상품 정보와 배달 주소를 한 번에 받습니다.

필드	타입	역할
productId	int	상품 ID
quantity	int	수량
price	Long	단가
address	String	배달 주소

OrderResponse — 저장된 Order의 필드(`id`, `userId`, `productId`, `quantity`, `price`, `status`, `createdAt`, `updatedAt`)를 그대로 펼쳐 반환합니다. `from(Order)` 정적 팩토리로 Order 엔티티에서 생성합니다.

전체 코드는 GitHub [ex01/order/.../orders/](#) 참조.

2.6.4 보상 트랜잭션 - 실패하면 되돌린다

엔티티가 준비되었으니, 이제 주문 서비스의 진짜 역할을 살펴봅니다. 주문 서비스는 상품 서비스와 배달 서비스를 **직접 호출**합니다. 그런데 서비스를 호출하면 반드시 이 질문과 마주합니다.

"재고는 줄었는데, 배달 생성이 실패하면 어떻게 하죠?"

이미 발송한 택배를 취소하려면 받는 사람에게 직접 "되돌려달라"고 요청해야 합니다. MSA의 보상도 마찬가지입니다.

단일 서비스였다면 DB 트랜잭션이 자동으로 롤백해줍니다. 하지만 이미 상품 서비스에 보낸 HTTP 요청은 되돌릴 수 없습니다. 우리가 직접 "재고를 다시 늘려줘"라는 HTTP 요청을 보내야 합니다. 이것이 **보상 트랜잭션**입니다. 챕터 1에서 소개한 대로, 주문 서비스가 직접 각 서비스를 호출하고 실패 시 역순으로 보상하는 방식입니다.

2.6.5 보상 트랜잭션 설계

핵심 아이디어는 **진행 상태를 플래그로 추적**하는 것입니다. 주문 PENDING 저장 → 재고 감소 → 배달 생성 → 주문 완료 순서로 진행되며, `productDecreased` 와 `deliveryCreated` 두 boolean 변수가 각각 재고 감소·배달 생성의 성공 여부를 기록합니다.

예외가 발생하면 이 플래그를 보고 켜진 단계만 골라 역순으로 외부 서비스를 보상하고, 마지막 `throw` 로 `@Transactional` 이 주문 row 자체를 롤백합니다.

정상 흐름은 다음 절의 그림 2-1, 실패 흐름은 그림 2-2에서 시퀀스로 살펴봅니다.

2.6.6 주문 성공 시

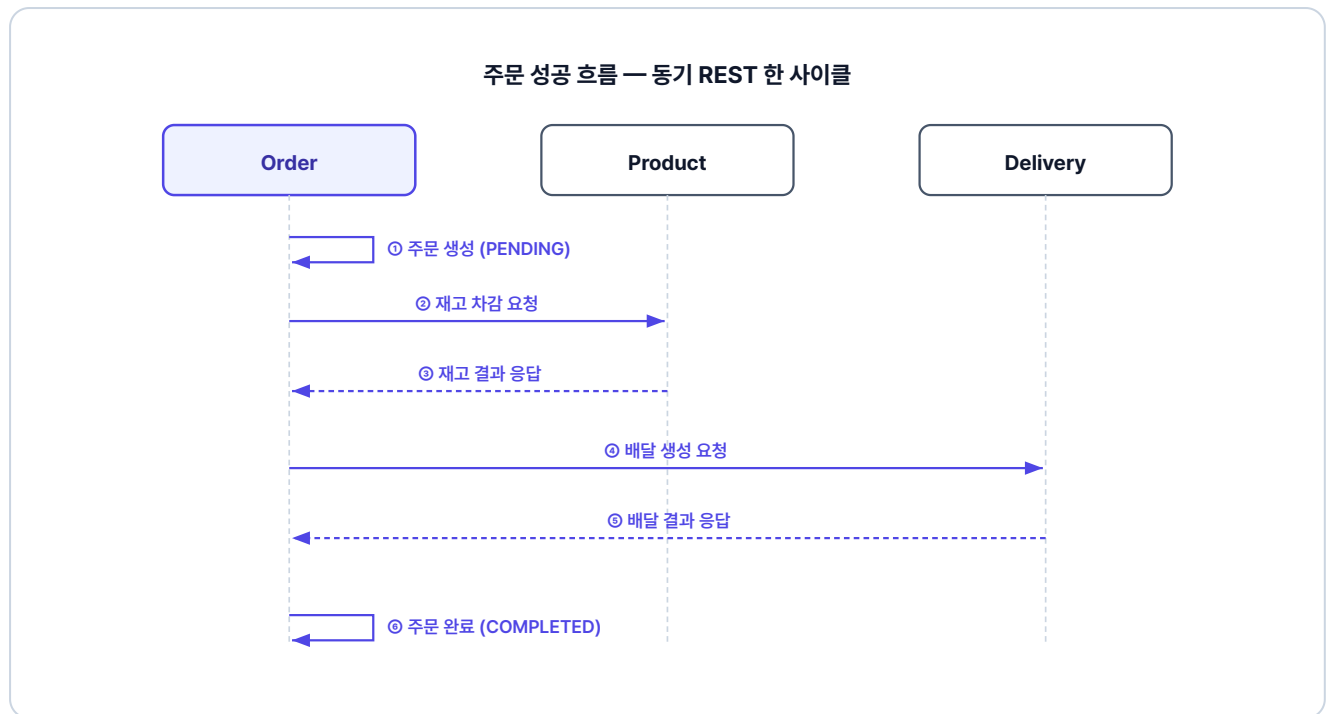


그림 2-1. 주문 성공 흐름

2.6.7 주문 실패 시

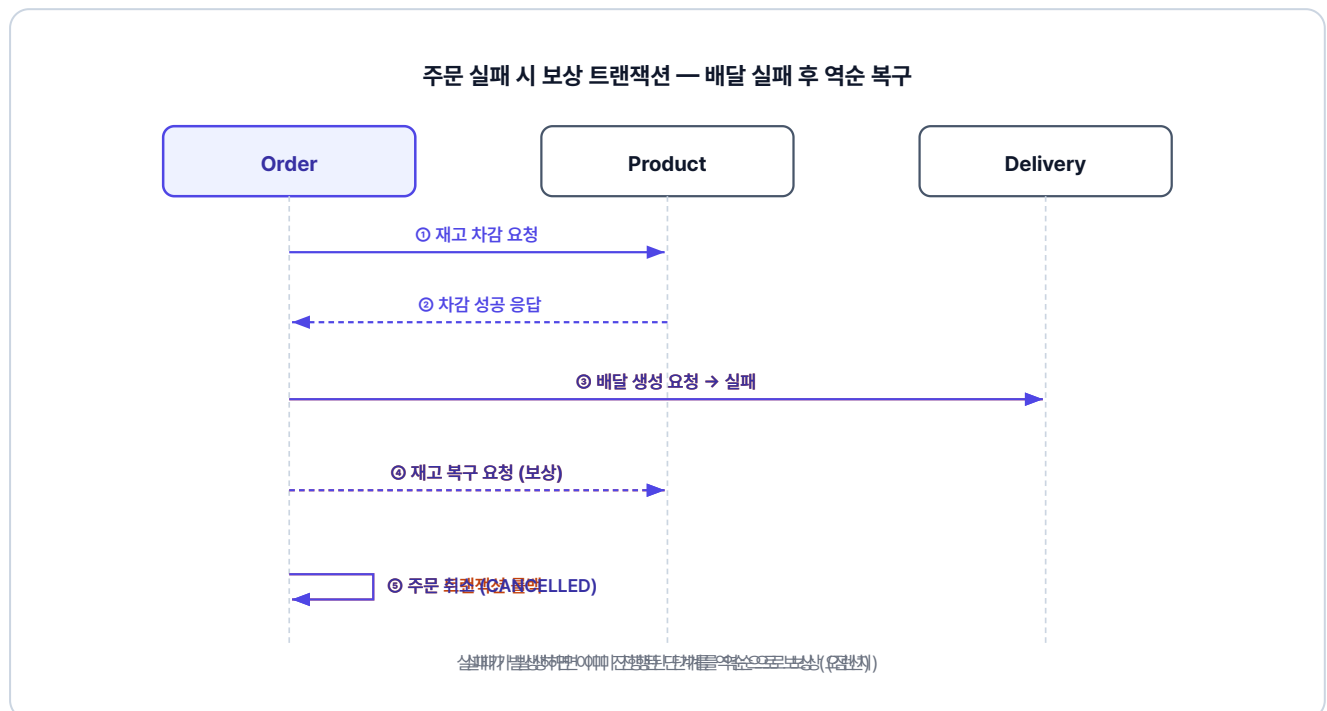


그림 2-2. 주문 실패 시 보상 트랜잭션 흐름

2.6.8 RestClient - JWT 헤더를 실어 다른 서비스를 호출하다

주문 서비스가 다른 서비스를 호출할 때 필요한 RestClient를 먼저 설정합니다. 인터셉터는 HTTP 요청이 나갈 때 자동으로 JWT 토큰을 챙겨 붙여주는 장치입니다. RestClient에 인터셉터 (`ClientHttpRequestInterceptor`)를 등록하면 외부 API를 호출할 때 Authorization 헤더가 자동으로 전달되어 하위 서비스의 JWT 인증을 통과합니다. 택배를 보낼 때 보내는 사람 정보를 매번 적는 대신, 스티커를 자동으로 붙여주는 것과 비슷합니다.

`core/config/RestClientConfig.java` 를 열고 아래 코드를 작성합니다.

실습 1 core/config/RestClientConfig.java. JWT 헤더 전달 인터셉터

```
@Configuration
public class RestClientConfig {

    @Bean
    public RestClient.Builder restClientBuilder() {
        ClientHttpRequestInterceptor authForwardingInterceptor = (request, body, execution) -> { // JWT 전달 인터셉터
            ServletRequestAttributes attributes =
                (ServletRequestAttributes) RequestContextHolder.getRequestAttributes(); // 현재 요청 정보 가져오기
            if (attributes != null) {
                String authorization = attributes.getRequest().getHeader("Authorization"); // 원본 요청의 JWT 꺼내기
                if (authorization != null) {
                    request.getHeaders().add("Authorization", authorization); // 나가는 요청에 JWT 실어 보내기
                }
            }
            return execution.execute(request, body);
        };

        return RestClient.builder().requestInterceptor(authForwardingInterceptor); // 인터셉터 등록한 빌더 반환
    }
}
```

2.6.9 ProductClient와 DeliveryClient

RestClient 설정이 끝났으면, 각 서비스를 호출하는 클라이언트를 만듭니다. OrderService는 클라이언트의 메서드만 호출하면 되고, HTTP 통신 방식을 알 필요가 없습니다.

baseUrl의 `product-service`, `delivery-service` 는 **Docker Compose**가 만들어주는 내부 주소입니다. 같은 Docker Compose 안에서는 서비스 이름만으로 서로 통신할 수 있습니다. Docker Compose 설정은 2.7절에서 다룹니다.

상품 서비스의 재고 감소·복구 API를 호출합니다.

`adapter/ProductClient.java` 를 열고 아래 코드를 작성합니다.

실습 2 adapter/ProductClient.java. 상품 서비스 호출 클라이언트

```
@Component
public class ProductClient { // 상품 서비스 호출 클라이언트

    private final RestClient restClient;

    public ProductClient(RestClient.Builder restClientBuilder) {
        this.restClient = restClientBuilder
            .baseUrl("http://product-service:8082") // 서비스 이름으로 통신 (Docker Compose가 DNS 역할)
            .build();
    }

    public void decreaseQuantity(ProductRequest requestDTO) { // 재고 감소 요청
        restClient.put()
            .uri("/api/products/{productId}/decrease", requestDTO.productId())
            .body(requestDTO)
            .retrieve()
            .toBodilessEntity();
    }

    public void increaseQuantity(ProductRequest requestDTO) { // 재고 복구 요청 (보상 트랜잭션용)
        restClient.put()
            .uri("/api/products/{productId}/increase", requestDTO.productId())
            .body(requestDTO)
            .retrieve()
            .toBodilessEntity();
    }
}
```

배달 서비스의 배달 생성·취소 API를 호출합니다.

`adapter/DeliveryClient.java` 를 열고 아래 코드를 작성합니다.

```
@Component
public class DeliveryClient { // 배달 서비스 호출 클라이언트

    private final RestClient restClient;

    public DeliveryClient(RestClient.Builder restClientBuilder) {
        this.restClient = restClientBuilder
            .baseUrl("http://delivery-service:8084") // 서비스 이름으로 통신 (Docker
Compose가 DNS 역할)
            .build();
    }

    public void createDelivery(DeliveryRequest requestDTO) { // 배달 생성 요청
        restClient.post()
            .uri("/api/deliveries")
            .body(requestDTO)
            .retrieve()
            .toBodilessEntity();
    }

    public void cancelDelivery(int orderId) { // 배달 취소 요청 (보상 트랜잭션용)
        restClient.put()
            .uri("/api/deliveries/{orderId}", orderId)
            .retrieve()
            .toBodilessEntity();
    }
}
```

선배 : "이제 진짜 핵심이에요. 각 단계마다 플래그를 꽂아두고, 실패하면 플래그 보고 되돌려요."

재고를 줄였으면 다시 늘리고, 배달을 만들었으면 취소하고... 그 말이 이거였구나.

2.6.10 OrderService - 보상 트랜잭션의 핵심

이제 이번 챕터의 하이라이트입니다. 보상 트랜잭션 패턴을 직접 구현합니다. 코드를 읽을 때

`productDecreased` 와 `deliveryCreated` 두 플래그를 추적하면서 읽어보세요. 단계가 성공할 때마다 플래그를 `true` 로 올려두고, catch 블록에서는 커져 있는 플래그만 골라 역순으로 되돌립니다.

`orders/OrderService.java` 를 열고 아래 메서드를 작성합니다.

```
@Transactional
public OrderResponse createOrder(int userId, int productId, int quantity, Long price,
String address) {
    // 보상트랜잭션을 위한 변수 선언
    boolean productDecreased = false;
    boolean deliveryCreated = false;

    // 보상트랜잭션에서 id를 전달해야해서 상위로 빼둠
    Order createdOrder = null;

    try {
        // 1. 주문 생성
        createdOrder = orderRepository.save(Order.create(userId, productId, quantity,
price));

        // 2. 상품 재고 차감
        productClient.decreaseQuantity(new ProductRequest(productId, quantity, price));
        productDecreased = true;

        // 3. 배달 생성 (어댑터)
        deliveryClient.createDelivery(new DeliveryRequest(createdOrder.getId(), address));
        deliveryCreated = true;

        // 4. 주문 완료
        createdOrder.complete();
        return OrderResponse.from(createdOrder);

    } catch (Exception e) {
        // 배달 취소
        if (deliveryCreated) {
            deliveryClient.cancelDelivery(createdOrder.getId());
        }

        // 재고 복구
        if (productDecreased) {
            productClient.increaseQuantity(new ProductRequest(productId, quantity, price));
        }

        throw new Exception500("주문 생성 중 오류가 발생했습니다: " + e.getMessage());
    }
}
```

참고로 catch 마지막의 `throw` 로 `@Transactional` 이 주문 트랜잭션까지 함께 롤백합니다. 외부 서비스는 보상 호출, 주문 row는 트랜잭션 롤백으로 정리되며, 실제 동작은 2.7.5 시나리오 3에서 확인합니다.

OrderService의 나머지 두 메서드입니다. `findById` 는 주문 한 건을 조회합니다. `cancelOrder` 는 `createOrder`의 역과정으로, 재고 복구(`increaseQuantity`) → 배달 취소(`cancelDelivery`) → 주문 상태 변경(`cancel()`) 순서로 처리합니다.

orders/OrderService.java. cancelOrder - 보상 호출 흐름

```
// findById(int orderId) - 단순 조회 메서드, 생략

@Transactional
public OrderResponse cancelOrder(int orderId) {
    Order findOrder = orderRepository.findById(orderId)
        .orElseThrow(() -> new Exception404("주문을 찾을 수 없습니다."));
    if (findOrder.getStatus() == OrderStatus.CANCELLED) {
        throw new Exception400("주문이 이미 취소되었습니다.");
    }
    productClient.increaseQuantity( // 재고 복구
        new ProductRequest(findOrder.getProductId(), findOrder.getQuantity(), findOrder.getPrice())
    );
    deliveryClient.cancelDelivery(orderId); // 배달 취소
    findOrder.cancel(); // 주문 상태 변경
    return OrderResponse.from(findOrder);
}
```

2.7 Docker Compose - 네 개의 서비스를 한 번에 실행하다

코드가 완성되었습니다. 이제 직접 실행하여 보상 트랜잭션이 작동하는 것을 눈으로 확인해 봅니다.

2.7.1 서비스 실행

각 서비스의 Dockerfile은 동일한 구조로, JDK 21 베이스 이미지에서 `gradle bootJar` 로 빌드해 실행합니다. 전체 내용은 GitHub 레포의 각 서비스 `Dockerfile` 참조.

ex01 디렉토리의 `docker-compose.yml` 로 4개 서비스를 한 번에 실행합니다.

4개 서비스 구조가 동일하므로, 주문 서비스만 예시로 보여줍니다.

docker-compose.yml. 4개 서비스 동시 실행

```
services:
  order-service:                # 주문 서비스 정의
    build:
      context: ./order          # 빌드할 소스 경로
    ports:
      - "8081:8081"            # 호스트:컨테이너 포트 매핑
    networks:
      - msa-network             # 같은 네트워크에 연결해야 서비스 이름으로 통신 가능

# user-service(8083), product-service(8082), delivery-service(8084) 동일 패턴 (Github
# 에서 확인 가능합니다)

networks:
  msa-network:                  # 4개 서비스를 하나로 묶는 가상 네트워크
```

`msa-network` 로 묶여 있기 때문에, 컨테이너끼리는 서비스 이름(예: `http://product-service:8082`)으로 통신합니다.

프로젝트가 위치한 폴더로 이동 후, 터미널에서 Docker Compose로 4개 서비스를 한 번에 빌드하고 실행합니다.

터미널 Docker Compose 실행

```
cd ex01
docker compose up
```

실행이 완료되면 각 서비스에 접근할 수 있습니다.

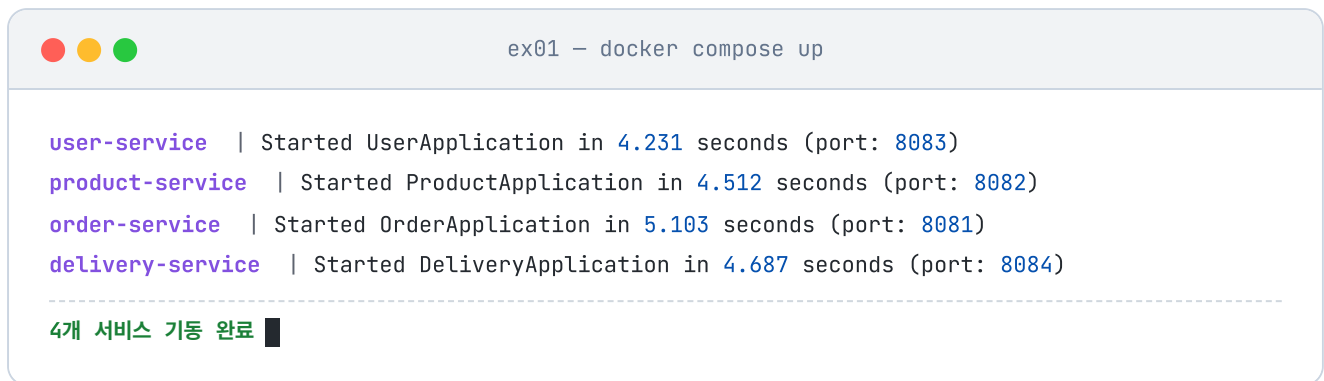
서비스	주소
주문 서비스	http://localhost:8081
상품 서비스	http://localhost:8082
회원 서비스	http://localhost:8083
배달 서비스	http://localhost:8084

2.7.2 사전 준비

실행 시나리오를 따라하려면 두 가지가 필요합니다.

Docker Desktop

Docker가 설치되어 있어야 합니다. <https://www.docker.com/products/docker-desktop/> 에서 설치하세요. Docker Desktop을 실행하고 화면 하단에 "Engine running"이 표시되면 준비 완료입니다. 터미널에서 `docker compose up` 을 실행합니다. 처음 실행 시 이미지 빌드에 5~10분이 소요될 수 있습니다. 터미널이 멈춘 것처럼 보여도 정상이니 기다려 주세요. 빌드 진행 상황은 `docker compose logs -f [서비스명]` 으로 확인할 수 있습니다.



```
ex01 - docker compose up

user-service | Started UserApplication in 4.231 seconds (port: 8083)
product-service | Started ProductApplication in 4.512 seconds (port: 8082)
order-service | Started OrderApplication in 5.103 seconds (port: 8081)
delivery-service | Started DeliveryApplication in 4.687 seconds (port: 8084)
-----
4개 서비스 기동 완료
```

그림 2-3. Docker Compose 실행 결과

API 테스트 도구 — Hoppscotch

서비스가 실행되면 API를 호출하여 결과를 확인해야 합니다. 이 책에서는 Hoppscotch(<https://hoppscotch.io/>)를 사용합니다. localhost로 요청을 보내려면 Chrome 웹 스토어에서 **Hoppscotch Browser Extension**을 설치합니다. 설치 후 Hoppscotch 화면 하단의 인터셉터 설정에서 "**Browser Extension**" 을 선택하세요.

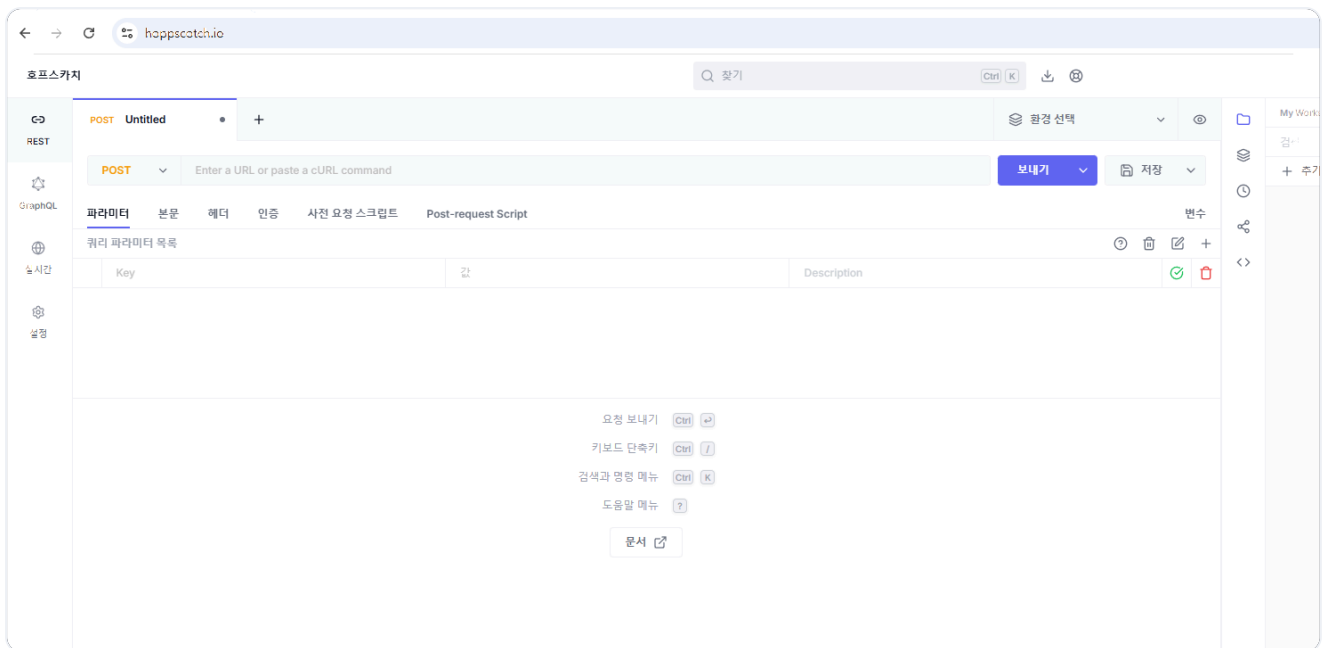


그림 2-4. Hoppscotch 화면

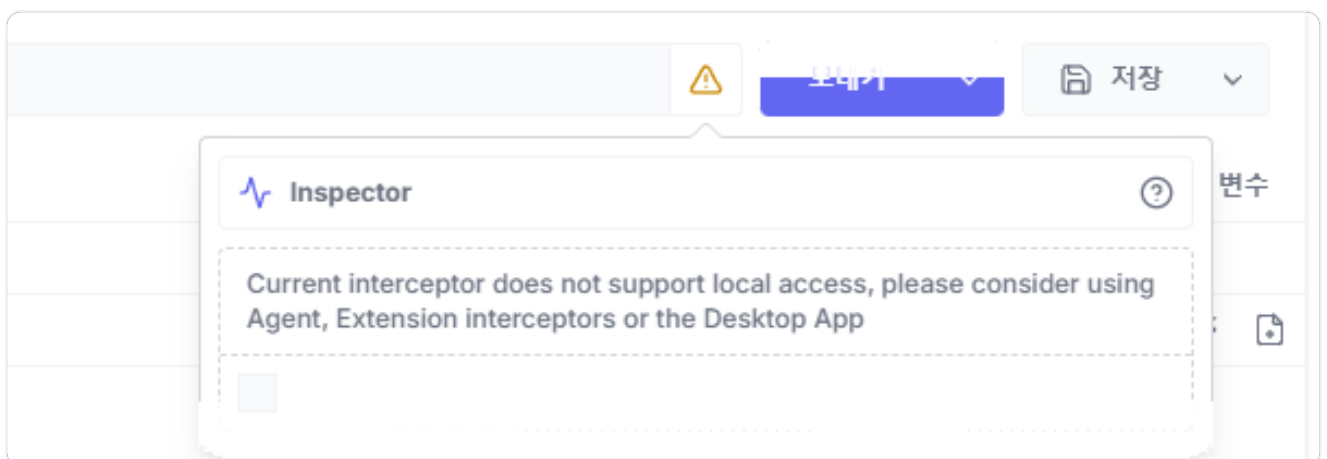


그림 2-5. Browser Extension 인터셉터 설정

오픈이 : "전부 띄웠는데, 진짜 되돌아가는지 어떻게 확인하죠?"

선배 : "품질 상품을 주문해 봐요. 그러면 작성한 보상 코드가 돌아가는 걸 눈으로 볼 수 있어요."

2.7.3 시나리오 1: 정상 주문

먼저 로그인하여 JWT 토큰을 받습니다. 이때 콘텐츠 종류(Content-Type)를

`application/json` 으로 설정해야 합니다. 이 헤더는 서버에게 "내가 보내는 데이터는 JSON 형식이다"라고 알리는 역할을 합니다.

```
{
  "username": "ssar",
  "password": "1234"
}
```



받은 토큰을 Hoppscotch의 인증 > 인증 유형(Bearer) 항목의 토큰 필드에 넣습니다.



MacBook Pro(상품 ID 1, 재고 10개)를 1개 주문합니다. 요청 바디는 상품 정보(`productId`, `quantity`, `price`)와 배달 주소(`address`)를 한 번에 담습니다.

POST `http://localhost:8081/api/orders`

```
{
  "productId": 1,
  "quantity": 1,
  "price": 2500000,
  "address": "Addr 4"
}
```

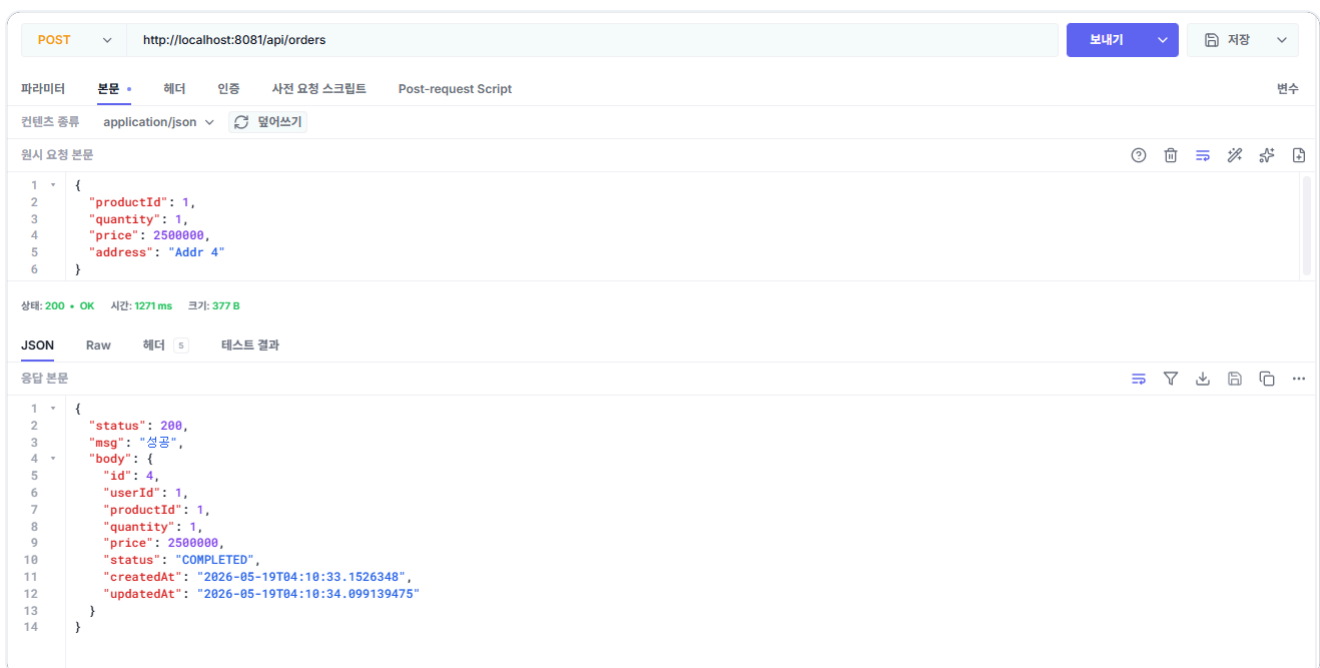


그림 2-8. 주문 생성 API 호출 결과

주문이 성공하면 상품 서비스에서 재고가 10 → 9로 줄어들고, 배달 서비스에 배달이 생성됩니다.

GET `http://localhost:8082/api/products/1`

상태: 200 • OK 시간: 87 ms 크기: 272 B

JSON Raw 헤더 5 테스트 결과

응답 본문

```

1  {
2    "status": 200,
3    "msg": "성공",
4    "body": {
5      "id": 1,
6      "productName": "MacBook Pro",
7      "quantity": 9,
8      "price": 2500000
9    }
10 }

```

그림 2-9. 재고 감소 확인

GET <http://localhost:8084/api/deliveries/4>

GET http://localhost:8084/api/deliveries/4 보내기 저장

파라미터 본문 헤더 인증 사전 요청 스크립트 Post-request Script 변수

컨텐츠 종류 application/json 덮어쓰기

원시 요청 본문

1 원시 요청 본문

상태: 200 • OK 시간: 35 ms 크기: 350 B

JSON Raw 헤더 5 테스트 결과

응답 본문

```

1  {
2    "status": 200,
3    "msg": "성공",
4    "body": {
5      "id": 4,
6      "orderId": 4,
7      "address": "Addr 4",
8      "status": "COMPLETED",
9      "createdAt": "2026-05-19T04:10:33.938551",
10     "updatedAt": "2026-05-19T04:10:34.034665"
11   }
12 }

```

그림 2-10. 배달 생성 확인

2.7.4 시나리오 2: 재고 부족

품질 상품인 iPhone 15(상품 ID 2, 재고 0)를 주문해 보겠습니다. 첫 번째 단계(재고 차감)에서 바로 실패하므로 보상할 작업이 없어 즉시 에러가 반환됩니다.

```
POST http://localhost:8081/api/orders
```

```
{
  "productId": 2,
  "quantity": 1,
  "price": 1300000,
  "address": "Addr 4"
}
```



그림 2-11. 재고 부족 시 에러 응답

2.7.5 시나리오 3: 주소 누락

이번에는 주소를 빈 문자열로 보내 보겠습니다. 주문 자체는 재고 차감까지 진행되지만, 배달 서비스에서 주소가 없으므로 실패합니다. 이때 보상 트랜잭션이 작동하여 차감된 재고가 복구되고, 주문 row는 트랜잭션 롤백으로 처음부터 없던 일처럼 DB에서 사라집니다.

```
POST http://localhost:8081/api/orders
```

```
{
  "productId": 1,
  "quantity": 1,
  "price": 2500000,
  "address": ""
}
```

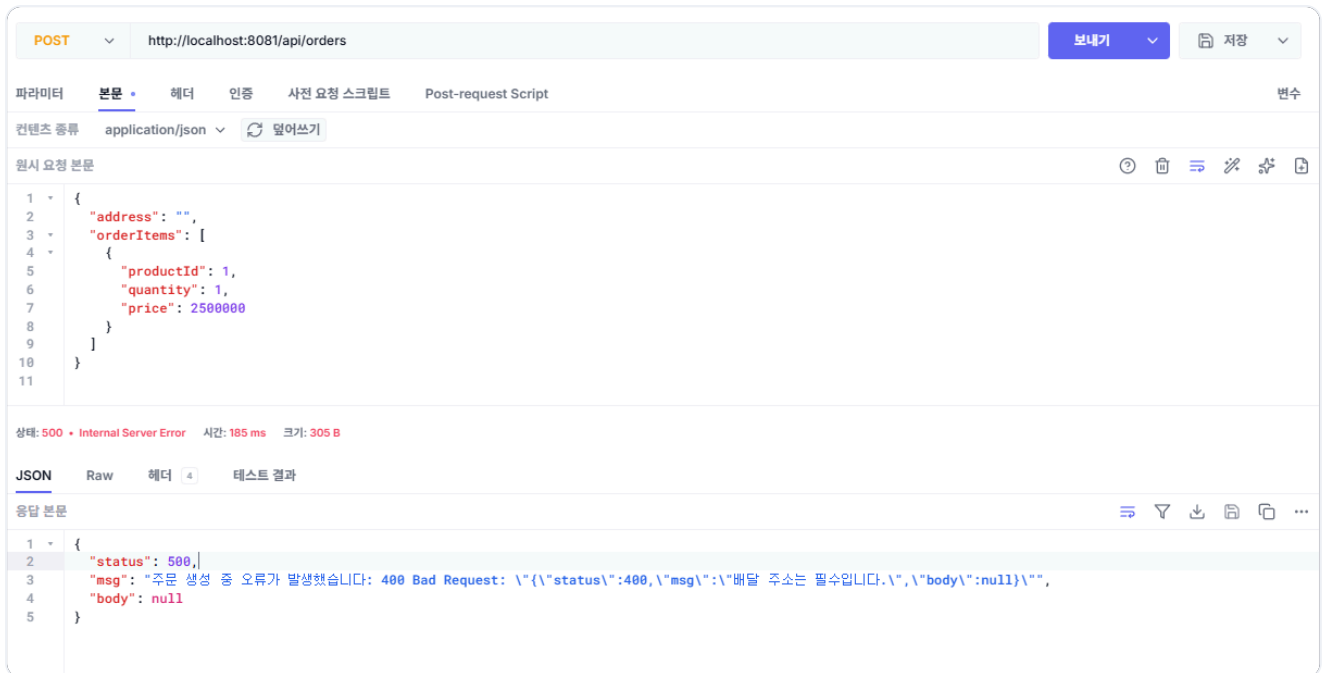


그림 2-12. 주소 누락 시 에러 응답

그리고 재고가 원복되었는지 확인합니다.

GET http://localhost:8082/api/products/1



그림 2-13. 재고 원복 확인

테스트가 끝났으면 실행 중인 컨테이너를 정리합니다.

```
docker compose down
```

이 명령어를 실행하면 docker compose up으로 띄운 모든 컨테이너가 종료되고 제거됩니다.

품질 상품을 주문하니까 에러가 나고, 재고가 자동으로 복구됐다. 된다!

선배 : "잘 됐네요. 근데 한 가지 더 생각해 봐요. 상품 서비스가 느려지면 주문 서비스는 어떻게 될까요?"

...아. 전화를 걸고 기다리니까, 상대가 느리면 나도 느려지는 거잖아.

보상 트랜잭션 덕분에 데이터 일관성을 유지할 수 있었습니다. 하지만 이 구조에는 눈에 잘 보이지 않는 문제가 몇 가지 있습니다. 모든 서비스 호출이 동기적이라 상품 서비스가 1초 걸리면 주문 서비스도 1초를 기다립니다. 보상 트랜잭션 코드도 비즈니스 로직과 섞이면서 try-catch 중첩이 깊어집니다.

실패한 주문은 트랜잭션 롤백으로 row 자체가 사라지므로 "어떤 주문이 왜 실패했는지" 이력도 남지 않습니다. 게다가 지금은 로컬에서만 실행되니, 실제 운영 서버에 올리려면 배포 방법을 별도로 고민해야 합니다.

이것만은 기억하자

- MSA에서 분산 트랜잭션은 자동으로 풀리지 않습니다. 실패를 감지한 쪽이 직접 보상 호출을 보내야 합니다.
- 4개 서비스를 REST로 연결하고, JWT 인증 헤더를 RestClient 인터셉터로 자동 전달했습니다.
- 플래그 기반 보상 트랜잭션으로 외부 서비스(재고와 배달)를 원상복구했습니다.
- 이 구조의 한계는 다음 챕터 운영 환경에서, 그다음 챕터 비동기 전환에서 차례로 해소됩니다.

코드 구조가 복잡해지는 것도 문제지만, 더 근본적인 문제는 **운영 가능한 형태로 만들어야 한다**는 것입니다. 다음 챕터에서는 코드 구조를 Clean Architecture로 개선하고, Kubernetes로 배포하는 방법을 배웁니다.